

PL101 — Programming Languages

Final Project Documentation

2nd Semester, A.Y. 2025–2026

Expense Tracker — Simple CRUD Application

Programming Language:	Visual Basic .NET (VB.Net)
Database:	MySQL via phpMyAdmin
Application Type:	Desktop-based (Windows Forms)
Subject:	PL101 — Programming Languages
School Year:	2nd Semester, A.Y. 2025–2026

1. Project Overview

The **Expense Tracker** is a desktop-based CRUD application developed using Visual Basic .NET and MySQL as the backend database. It allows users to record, view, update, and delete personal expense entries. The application was developed as the final project for PL101, demonstrating key programming language concepts through a functional and practical system.

The system manages the following expense information:

- **Title** — Name or description of the expense
- **Amount** — Cost of the expense in decimal format
- **Category** — Type of expense: Food, Transport, Bills, Shopping, or Others
- **Date** — Date when the expense was made
- **Notes** — Optional additional details about the expense

2. CRUD Functionality

Operation	Description	How It Works
CREATE	Add a new expense record	Fill in Title, Amount, Category, Date, and Notes then click ADD
READ	View all expense records	All records are loaded automatically in the DataGridView on startup
UPDATE	Modify an existing expense	Click a row in the grid, edit the fields, then click UPDATE
DELETE	Remove an expense record	Click a row in the grid, then click DELETE and confirm the prompt

3. Database Design

Database: `expense_tracker` | Table: `tblexpense`

Column	Type	Description
--------	------	-------------

id	INT, AUTO_INCREMENT	Primary key, auto-generated
title	VARCHAR(100)	Expense name/description
amount	DECIMAL(10,2)	Cost of the expense
category	VARCHAR(50)	Expense category
date_added	DATE	Date of the expense
notes	VARCHAR(255)	Optional additional details

SQL to create the table:

```
CREATE TABLE tblexpense (
  id INT NOT NULL AUTO_INCREMENT,
  title VARCHAR(100) NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  category VARCHAR(50) NOT NULL,
  date_added DATE NOT NULL,
  notes VARCHAR(255),
  PRIMARY KEY (id)
);
```

4. Programming Language Concepts Applied

This section explains how the Expense Tracker demonstrates at least three Programming Language concepts discussed in PL101.

4.1 Error Handling and Exception Handling

VB.Net uses structured exception handling through **Try...Catch...End Try** blocks. In the Expense Tracker, every database operation (ADD, UPDATE, DELETE, READ) is wrapped in a Try-Catch block. This prevents the application from crashing when unexpected errors occur, such as a lost database connection or invalid SQL. The Catch block captures the exception object and displays a meaningful error message to the user.

Example from the ADD operation:

```
Try
    If comm.ExecuteNonQuery() = 1 Then
        MessageBox.Show("Expense added successfully!")
    Else
        MessageBox.Show("Failed to add expense.")
    End If
Catch ex As Exception
    MessageBox.Show("Error: " & ex.Message)
End Try
```

This demonstrates **exception handling** — a core PL concept where the language provides a mechanism to detect and respond to runtime errors gracefully rather than terminating abruptly.

4.2 Type Systems and Static Typing

VB.Net is a **statically-typed** language, meaning all variable types are declared and checked at compile time. In the Expense Tracker, variables are explicitly declared with their types using **Dim**, and the compiler enforces type rules before the program runs. This catches type errors early and improves code reliability.

Examples of static typing in the project:

- Dim conn As MySqlConnection — connection object typed explicitly
- Dim amt As Decimal — amount stored as Decimal, not String
- Dim dt As New DataTable() — data table explicitly typed
- Dim comm As New MySqlCommand(...) — command object with specific type

Additionally, **Decimal.TryParse()** is used for type validation — ensuring the Amount field contains a valid decimal number before attempting to save it to the database. This is an example of **type safety** and **defensive programming**.

```

Dim amt As Decimal
If Not Decimal.TryParse(txtAmount.Text, amt) Then
    MessageBox.Show("Amount must be a valid number.")
    Return False
End If

```

4.3 Binding and Scope

VB.Net enforces clear **variable binding and scope rules**. In the Expense Tracker, variables are declared at different scope levels depending on their purpose:

Variable	Scope	Reason
conn, adaptr	Class-level (module scope)	Shared across all methods — connection is reused
insertQuery, updateQuery	Local (method scope)	Only needed inside one specific Sub
comm	Local (method scope)	Command object used only within its operation
amt	Local (function scope)	Temporary variable for validation only

The **conn** and **adaptr** variables are bound at the class level so they persist for the lifetime of the form. Local variables like **insertQuery** are bound only within their subroutine and are released when the method returns — demonstrating proper use of scope to manage resources efficiently.

5. Code Structure and Organization

The source code is organized into clearly separated, well-named subroutines. Each method has a single responsibility, making the code readable and maintainable.

- `loadCategories()` — Populates the ComboBox with predefined expense categories on startup
- `displayData()` — Fetches all records from MySQL and binds them to the DataGridView
- `clearData()` — Resets all input fields to their default empty/initial state
- `isValid()` — Validates user inputs before any database operation is performed
- `Form1_Load()` — Entry point — calls `loadCategories()` and `displayData()` on startup
- `btnADD_Click()` — Handles the ADD button — validates input then inserts a new record
- `UPDATE_Click()` — Handles the UPDATE button — modifies the selected record in the database
- `Button3_Click()` / `DELETE` — Handles the DELETE button — removes the selected record after confirmation
- `DataGridView1_CellContentClick()` — Loads selected row data into the input fields for editing

6. Input Validation and Defensive Programming

The application applies several defensive programming techniques to prevent invalid data from entering the database and to prevent crashes during runtime:

Validation	Description
Empty field check	Title must not be blank before saving
Numeric type validation	<code>Decimal.TryParse()</code> ensures Amount is a valid number
Positive value check	Amount must be greater than zero
Row selection check	UPDATE and DELETE require a record to be selected first
Null row guard	CellClick handler checks row IsNot Nothing before reading cells
Connection state check	<code>conn.State</code> is verified before closing to avoid double-close errors
Confirmation dialog	DELETE asks for Yes/No confirmation before removing a record

7. System Requirements

Component	Requirement
Programming Language	Visual Basic .NET (VB.Net)
IDE	Microsoft Visual Studio
Framework	.NET Framework
Database	MySQL (via phpMyAdmin / XAMPP)
MySQL Connector	MySql.Data (NuGet Package)
Operating System	Windows

8. Summary

The Expense Tracker successfully demonstrates all required CRUD operations through a clean, functional desktop application built in VB.Net. The project applies core Programming Language concepts including exception handling, static typing and type safety, and variable binding and scope. The code is organized into single-purpose methods, uses proper input validation, and handles runtime errors gracefully. The application connects to a MySQL database and correctly performs Create, Read, Update, and Delete operations on expense records.

The simplicity of the system reflects the project goal: to demonstrate understanding of programming language concepts through a working, well-structured application.